

NoSQL Databases — Complete Revision Handbook

TG CPGET 2026 | Unit: NoSQL Databases

How to use this handbook

Each topic has: **Simple Explanation** → **Key Definitions/Points** → **Examples** → **(at the end) Practice Questions**. Read once for understanding, then revise only the *Key Points* boxes before the exam.

1. Why NoSQL?

Simple Explanation

For decades, Relational Databases (RDBMS) were the default choice for almost every application. But around the mid-2000s, companies like Google, Amazon, and Facebook started facing **huge volumes of data, fast-changing data structures**, and the need to run on **thousands of cheap servers** instead of one expensive server. Relational databases struggled with this because:

- They need a **fixed schema** (structure decided in advance).
- Scaling them means buying a **bigger, costlier machine** (vertical scaling) rather than adding more machines (horizontal scaling).
- Joins across many tables become very slow on huge, distributed data.

NoSQL ("Not Only SQL") databases were built to solve exactly these problems — flexible schema, easy horizontal scaling, and better handling of big, fast, varied data (the "3 Vs": **Volume, Velocity, Variety**).

Key Points

- **NoSQL = Not Only SQL** — a category of databases that are non-relational.
- Main drivers: **Impedance mismatch** (object-oriented app data vs relational tables), need for **cluster-friendly / horizontal scalability**, and handling of **large-scale, semi-structured/unstructured data**.
- NoSQL databases generally trade strict **ACID** (Atomicity, Consistency, Isolation, Durability) guarantees for **scalability and availability**, often following **BASE** (Basically Available, Soft state, Eventually consistent).
- **Polyglot Persistence**: using different databases (relational + NoSQL) for different parts of the same application, based on what each part needs.
- The rise of NoSQL is closely tied to the **CAP Theorem** (see Consistency section).

Examples

1. **Facebook** uses a column-family store (Cassandra) for its Messages feature due to huge write volume.
 2. **Amazon** built Dynamo (a key-value store) to keep its shopping cart always available, even during network failures.
 3. **Google** created Bigtable (column-family) to store and index the entire web.
 4. A social media app storing posts with different fields (some with images, some with polls, some with just text) — a fixed relational schema is painful; a document database handles this naturally.
 5. A gaming leaderboard needing millisecond reads/writes for millions of users — a key-value store like Redis is ideal.
-

2. Aggregate Data Models

Simple Explanation

In relational databases, related data is normalized and split across many tables (e.g., Customer, Order, OrderLine, Product). To get a complete picture of one order, you need **joins**.

NoSQL databases (except graph databases) usually take the opposite approach: they group all data that is **commonly accessed together** into a single unit called an **aggregate**. Think of an aggregate as a "bundle" — e.g., one JSON document containing a customer's order along with all its order lines, shipping address, and payment info — all in one place, no joins needed.

Key Definitions

- **Aggregate:** A collection of related objects treated as a single unit for data manipulation and consistency purposes (a term from Domain-Driven Design).
- **Aggregate-oriented databases:** Key-value, Document, and Column-family databases — all store and manipulate data as aggregates.
- **Graph databases** are the exception — they work with small nodes and relationships rather than big aggregates.
- **Atomic operations** in NoSQL are usually guaranteed **only within one aggregate**, not across multiple aggregates (unlike RDBMS transactions across tables).

Examples

1. **E-commerce Order Aggregate:** One document holds `{order_id, customer_name, items: [...], shipping_address, total}` — everything needed to display or process the order in one read.

2. **Relational vs Aggregate:** In SQL, displaying an order needs `JOIN` across `Orders`, `OrderItems`, `Customers`. In a document DB, it's one `find()` call.
 3. **Blog Aggregate:** A blog post document containing the post text plus an embedded array of comments — no separate "Comments" table needed.
 4. **Downside example:** If you need to find "all orders containing Product X" across thousands of order-aggregates, this is less efficient than a relational query with proper indexing, since data is duplicated/embedded rather than normalized.
 5. **Banking counter-example:** Aggregate orientation can be tricky when you need atomic transactions across two aggregates (e.g., transferring money between two account aggregates) — this needs special handling.
-

3. More Details on Data Models

Simple Explanation

There are four broad "families" of NoSQL data models, each suited to different problems:

Model	Stores data as	Best for
Key-Value	Value stored/retrieved by unique key (opaque to the DB)	Simple lookups, caching, session storage
Document	Semi-structured documents (JSON/XML/BSON) with visible internal structure	Content management, catalogs, user profiles
Column-Family	Rows with dynamic columns, grouped into "column families"	Large-scale, sparse, write-heavy data
Graph	Nodes + Edges + Properties	Highly connected data: social networks, recommendations

Key Points

- Key-Value and Document databases are similar; the key difference is that **document databases can see/query inside the value**, while key-value stores treat the value as an opaque blob.
- **Column-family** stores organize data as **rows and dynamic columns grouped into families** — good middle ground between key-value simplicity and relational structure.
- **Graph databases** are optimized for traversing **relationships** — something relational joins and aggregate-oriented models both do poorly at scale.

- Choice of data model depends on the **access pattern** of the application, not just the data shape.

Examples

1. **Key-Value:** Redis storing (`session_id → user_session_data`).
 2. **Document:** MongoDB storing a product catalog entry with nested attributes (`{name, price, specs: {color, size}}`).
 3. **Column-Family:** Cassandra storing sensor time-series data — each row is a sensor, columns are timestamps.
 4. **Graph:** Neo4j modeling "Person -[FRIENDS_WITH]-> Person" and "Person -[LIKES]-> Page" for a social network.
 5. **Mixed real system:** An e-commerce platform might use Document DB for product catalog, Key-Value for cart/session, and Graph DB for "customers who bought this also bought..." recommendations.
-

4. Distribution Models

Simple Explanation

NoSQL databases are designed to run across **many machines (a cluster)** rather than one big server. There are two main techniques used, often combined:

1. **Sharding** — splitting *different* data across different servers (each server holds a portion of the data) → increases capacity and throughput.
2. **Replication** — copying the *same* data onto multiple servers → increases availability and read performance.

Key Definitions & Formulas

- **Sharding:** Distributing data so each node stores only a subset. Ideally, data accessed together is on the same shard (data locality).
- **Replication** has two styles:
 - **Master-Slave Replication:** One master node handles all writes; slaves replicate from it and can serve reads. Simple, but master is a single point of failure for writes.
 - **Peer-to-Peer (Masterless) Replication:** All nodes are equal; any node can accept reads/writes. More complex conflict handling, but no single point of failure.
- **Combining Sharding + Replication:** Each shard itself is replicated across multiple nodes — a common production setup (e.g., Cassandra, MongoDB).
- Read/Write throughput scaling is roughly: more shards → more parallel writes; more replicas → more parallel reads (with some consistency trade-off).

Examples

1. **Sharding example:** A social network shards users A-M on Server 1 and N-Z on Server 2.
 2. **Master-Slave example:** MongoDB's traditional replica set — one primary handles writes, secondaries replicate.
 3. **Peer-to-Peer example:** Cassandra's ring architecture — every node is equal, and any node can accept a write.
 4. **Combined example:** Cassandra shards data using consistent hashing (partitioning) AND replicates each partition to N nodes (replication factor = 3, say).
 5. **Failure scenario:** In master-slave, if the master fails, a slave must be promoted (failover) before writes can resume — causing brief write unavailability.
-

5. Consistency

Simple Explanation

When data is copied across multiple nodes (replication), a key question arises: **if I write to one node, will I immediately see that update if I read from another node?** NoSQL systems make deliberate trade-offs here, famously captured by the **CAP Theorem**.

Key Definitions & Formulas

CAP Theorem

In a distributed system, you can only guarantee **2 out of 3** of:

- **C — Consistency:** every read gets the most recent write.
- **A — Availability:** every request gets a (non-error) response.
- **P — Partition Tolerance:** system keeps working despite network partitions (communication breakdown between nodes).

Since network partitions *will* happen in real distributed systems, practically the choice is between **CP** (consistent but may refuse requests during a partition) and **AP** (always available but may return stale data).

Types of Consistency

- **Strong (Strict) Consistency:** All reads return the latest write — like traditional RDBMS.
- **Eventual Consistency:** Given no new updates, all replicas will *eventually* converge to the same value — used by many AP-style NoSQL systems (e.g., Dynamo, Cassandra).

- **Read-your-writes Consistency:** A user always sees their own most recent write, even if others don't yet.
- **Write Consistency:** Deals with concurrent writes to the same data — handled via pessimistic (locking) or optimistic (conditional update/version check) concurrency control.

Quorum

A common technique to balance consistency and availability:

- **N** = number of replicas
- **W** = number of nodes that must acknowledge a write
- **R** = number of nodes contacted on a read
- **Rule for strong consistency:** $W + R > N$

Examples

1. **CP System:** HBase — favors consistency; may become unavailable during a network partition rather than serve stale data.
 2. **AP System:** Cassandra (default settings), Amazon Dynamo — always responds, but might return slightly outdated data temporarily.
 3. **Eventual Consistency in real life:** DNS updates — when you change a DNS record, it takes time to propagate worldwide, but eventually all resolvers show the new value.
 4. **Quorum example:** $N=3, W=2, R=2 \rightarrow W+R=4 > N=3$, so this configuration guarantees strong consistency.
 5. **Read-your-writes example:** After you post a comment on a social app, you always instantly see your own comment, even if some other users see it a moment later.
-

6. Version Stamps

Simple Explanation

When multiple users/processes might update the same piece of data at almost the same time, the database needs a way to detect **conflicting updates**. A **version stamp** is a marker attached to data that changes every time the data is updated, letting the system detect if the data changed since it was last read.

Key Definitions

- **Version Stamp:** A value (counter, timestamp, or hash) attached to a record that changes on every update — used to detect concurrent modification conflicts.
- Common implementations:
 - **Simple counter/sequence number:** increments by 1 with each update.

- **Timestamp-based:** uses the update time (works only if clocks are well synchronized).
- **Content hash:** a hash of the data itself; unlike counters, this doesn't require coordination.
- **Vector Clocks:** A list of (node, counter) pairs used in distributed, masterless systems (like Dynamo/Cassandra) to track causality — i.e., to know whether one version "happened before" another, or whether they conflict.

Examples

1. **Optimistic locking:** A document has `version: 5`. A client reads it, edits it, and tries to save with the condition "only update if version is still 5" — preventing the "lost update" problem.
 2. **Vector Clock example:** Node A updates a shopping cart making its clock `[A:1]`. Node B, unaware of A's update, updates the same cart to `[B:1]`. When these merge, the system detects a conflict (neither clock is "greater") and may ask the application to reconcile (this is exactly what happened with Amazon Dynamo's shopping cart).
 3. **HTTP ETags:** A web caching example of version stamps — a resource's ETag changes when the resource changes, letting clients detect staleness.
 4. **Timestamp risk example:** Two servers with unsynchronized clocks might wrongly order updates if using raw timestamps — a classic pitfall of timestamp-based version stamps.
 5. **CouchDB `_rev` field:** Every document update generates a new revision ID, which acts as its version stamp for conflict detection.
-

7. Map-Reduce

Simple Explanation

Map-Reduce is a programming model for processing **large datasets in parallel across a cluster**. Instead of moving huge amounts of data to one program, it moves small pieces of computation logic (the "map" and "reduce" functions) out to where the data lives, and combines results.

Key Definitions & Steps

1. **Map step:** A function is applied independently to each piece of input data (e.g., each document/record), producing intermediate **key-value pairs**.
2. **Shuffle/Group step:** All intermediate values with the same key are grouped together.
3. **Reduce step:** A function takes each group (same key + list of values) and combines/aggregates them into a final result.
4. **Partitioning:** Splitting input data across nodes so map tasks run in parallel.

5. **Combining function:** An optional step that pre-aggregates map output on the same node before shuffling, to reduce network traffic.
6. **Composing Map-Reduce calculations:** Complex analytics are built by chaining multiple map-reduce jobs (output of one becomes input of the next) — sometimes called a "map-reduce pipeline."

Examples

1. **Word Count** (classic example): Map emits `(word, 1)` for every word in a document; Reduce sums up counts per word to get total occurrences.
 2. **Sales aggregation:** Map emits `(product_category, sale_amount)` for each transaction; Reduce sums amounts per category to get total sales by category.
 3. **Log analysis:** Map extracts `(error_type, 1)` from server logs; Reduce counts frequency of each error type across millions of log lines.
 4. **Incremental Map-Reduce:** Running map-reduce only on newly added data (not the whole dataset again) to keep results updated efficiently — used in systems like CouchDB views.
 5. **Composed pipeline:** Job 1 computes average order value per customer (map-reduce #1); Job 2 takes that output and computes average order value per region (map-reduce #2 uses #1's output as input).
-

8. Key-Value Databases

Simple Explanation

The simplest NoSQL model: a giant dictionary/hash-map. You store a **value** against a unique **key**, and can only retrieve it by that exact key — like a coat-check ticket system. The database does not understand or query the contents of the value.

Key Points

- **Data model:** `key → value` (value is opaque — could be a string, blob, JSON, etc.)
- **Operations:** essentially `Get(key)`, `Put(key, value)`, `Delete(key)` — no complex queries.
- **Consistency:** often eventually consistent in distributed key-value stores.
- **Suitable for:** session storage, caching, shopping carts, user preferences, simple lookups requiring very high speed.
- **Not suitable for:** queries based on the *value's contents* (e.g., "find all users older than 30") or relationships between records.
- **Examples of real databases:** Redis, Riak, Amazon DynamoDB, Memcached.

1. **Session store:** `session_abc123 → {user_id: 42, logged_in_at: ...}` in Redis.
 2. **Caching:** Storing the result of an expensive computation as `query_hash → result` so repeat requests are instant.
 3. **Shopping cart:** `cart_user42 → [item1, item2, item3]`.
 4. **Leaderboard/counter:** `page_views_homepage → 193830` using atomic increment operations.
 5. **Config store:** `feature_flag_darkmode → true/false` for quick app-wide settings lookup.
-

9. Document Databases

Simple Explanation

A document database stores data as **documents** (usually JSON, BSON, or XML) — a step up from key-value stores because the database **understands the internal structure** and can query/index fields inside the document, without needing a fixed schema.

Key Points

- **Data model:** Self-describing documents, typically grouped into **collections** (roughly like tables, but schema-free).
- **Schema flexibility:** Different documents in the same collection can have different fields.
- **Querying:** Supports queries on nested fields, ranges, and even full-text search, unlike plain key-value stores.
- **Consistency:** Usually strong consistency at the single-document level; eventual consistency across replicas/shards.
- **Suitable for:** content management, catalogs, user profiles, blogs, event logging.
- **Examples of real databases:** MongoDB, CouchDB, Amazon DocumentDB.

Examples

1. **User profile document:**

```
json
{ "_id": 101, "name": "Anita", "email": "anita@mail.com", "hobbies": ["readi
```

2. **Product catalog document** with nested specs: `{ "name": "Phone X", "price": 25000, "specs": { "ram": "8GB", "color": "black" } }`.
 3. **Query example:** MongoDB query `db.products.find({"specs.ram": "8GB"})` — querying a nested field directly, something a key-value store cannot do.
 4. **Content management:** A CMS storing each blog article as one document, including embedded comments and tags.
 5. **Schema evolution example:** Adding a new field `"loyalty_points"` to only some user documents without altering every existing record or requiring a migration — demonstrating schema flexibility.
-

10. Column-Family Stores

Simple Explanation

Also called "wide column stores," these databases organize data into rows, but instead of a fixed set of columns (like RDBMS), each row can have a different, dynamic set of columns, grouped into **column families**. This design (popularized by Google's Bigtable) is optimized for **very large-scale, sparse data** and fast writes.

Key Points

- **Data model:** Rows identified by a **row key**; each row contains **column families**, and within each family, any number of columns can exist (even different per row).
- Data is often stored **column-wise on disk**, making it efficient to read/aggregate specific columns across many rows (good for analytics).
- **Two common design patterns:**
 - **Column-family as row-oriented:** Similar to a document store, one column family per entity.
 - **Column-family as columnar storage:** Great for aggregate computations over a single column across millions of rows.
- **Suitable for:** time-series data, sensor data, large-scale logging, systems needing very high write throughput.
- **Examples of real databases:** Apache Cassandra, HBase, Google Bigtable.

Examples

1. **Sensor data:** Row key = `sensor_id`; column family `readings` has columns as timestamps: `{10:00: 23°C, 10:05: 24°C, ...}` — different rows can have wildly different numbers of columns.
2. **Web crawling (Bigtable's original use case):** Row key = URL; column family stores different versions of the page content fetched at different times.

3. **User activity log:** Row key = `user_id`; columns are `login_time_1, login_time_2, ...` added dynamically as events happen.
 4. **Cassandra table example:** A `messages` column family where each row (per user) can have a different number of message columns, avoiding wasted space for users with few messages.
 5. **Analytics use case:** Fast aggregation of a single column (e.g., total clicks) across millions of rows without scanning unrelated columns — a benefit of columnar storage layout.
-

11. Graph Databases

Simple Explanation

Graph databases store data as a network of **nodes** (entities) connected by **edges** (relationships), with properties attached to both. Unlike aggregate-oriented models, graph databases are built specifically to make **traversing complex relationships** (friend-of-a-friend, shortest path, recommendations) extremely fast — something that requires expensive joins in relational databases.

Key Definitions

- **Node:** An entity/object (e.g., a Person, Product).
- **Edge (Relationship):** A connection between two nodes, which can be **directed** and **labeled** (e.g., `FRIENDS_WITH`, `PURCHASED`).
- **Properties:** Key-value attributes attached to nodes or edges (e.g., a `FRIENDS_WITH` edge might have a property `since: 2020`).
- **Traversal:** The operation of navigating from node to node via edges to answer relationship-based queries.
- **Query language example:** Cypher (used by Neo4j) — a declarative pattern-matching language for graphs.

Examples

1. **Social network:** `(Alice)-[:FRIENDS_WITH]->(Bob)-[:FRIENDS_WITH]->(Carol)` — used to find "friends of friends."
2. **Recommendation engine:** `((Customer)-[:BOUGHT]->(Product)<-[:BOUGHT]-(OtherCustomer)-[:BOUGHT]->(OtherProduct))` — powers "customers who bought this also bought..."
3. **Fraud detection:** Modeling transactions as a graph to detect unusual patterns, like many accounts funneling money to one account through short paths.
4. **Route/Navigation:** Modeling cities as nodes and roads as edges with `distance` properties to compute shortest paths (e.g., Google Maps-style routing).

5. **Knowledge graph:** Wikipedia's/Google's knowledge graph connecting entities like (Albert Einstein)-[:BORN_IN]->(Germany) for question-answering systems.
-

Quick Revision Table

Model	Real DB Examples	Data Unit	Best Use Case	Weak Point
Key-Value	Redis, DynamoDB	Opaque value by key	Caching, sessions	No query on value
Document	MongoDB, CouchDB	JSON/BSON document	Catalogs, CMS	Cross-document joins costly
Column- Family	Cassandra, HBase, Bigtable	Rows w/ dynamic columns	Time-series, huge writes	Complex modeling
Graph	Neo4j	Nodes + Edges	Relationships, recommendations	Not for huge aggregate scans

Practice Questions (TG CPGET-style, Multiple Choice)

Q1. Which of the following is the primary reason NoSQL databases emerged as an alternative to RDBMS? (a) Need for stronger ACID guarantees (b) Need for horizontal scalability and flexible schemas (c) Need for more complex SQL joins (d) Need for smaller data volumes

Q2. In the CAP theorem, if a distributed system chooses Consistency and Partition Tolerance (CP) over Availability, what is the likely behavior during a network partition? (a) It always returns stale data (b) It may refuse to respond to some requests to avoid inconsistent data (c) It shuts down permanently (d) It automatically becomes a key-value store

Q3. Which distribution technique involves storing different subsets of data on different nodes to increase capacity? (a) Replication (b) Sharding (c) Version stamping (d) Map-Reduce

Q4. A "quorum" read/write configuration with $N=5$, $W=3$, $R=3$ will: (a) Never guarantee consistency (b) Guarantee strong consistency since $W+R > N$ (c) Only work in master-slave systems (d) Require all 5 replicas to respond

Q5. Vector clocks are primarily used to: (a) Encrypt data across nodes (b) Detect and reason about causality/conflicts in concurrent updates (c) Speed up Map-Reduce jobs (d) Partition data into shards

Q6. In the Map-Reduce model, the step where intermediate key-value pairs with the same key are grouped together before final processing is called: (a) Partitioning (b) Combining (c) Shuffle/Group (d) Sharding

Q7. Which NoSQL data model is generally NOT considered "aggregate-oriented"? (a) Key-Value (b) Document (c) Column-Family (d) Graph

Q8. Which database type is best suited for a recommendation engine requiring "friends of friends" style traversal? (a) Key-Value store (b) Document store (c) Column-family store (d) Graph database

Q9. A retail company needs to store sensor readings from thousands of IoT devices, with each device reporting readings at irregular time intervals (some devices report thousands of readings, others just a few). Which NoSQL model is most appropriate? (a) Key-Value store (b) Document store (c) Column-Family store (d) Graph database

Q10. "Eventual Consistency" means: (a) The system never becomes consistent (b) All replicas will converge to the same value over time if no new updates occur (c) Every read always returns the latest write (d) Writes are rejected until all replicas agree

Answer Key

1. (b) | 2. (b) | 3. (b) | 4. (b) | 5. (b) | 6. (c) | 7. (d) | 8. (d) | 9. (c) | 10. (b)

Tip for revision: Focus on the Quick Revision Table + CAP theorem + Quorum formula ($W+R>N$) + the four data models' use cases — these are the highest-yield areas typically tested in PG entrance exams.